Unified PDAI Server

Consolidation Plan: Edition Configuration, Annotation System, and Karman Absorption

Gerd Graßhoff 2026-08-24

Table of contents

1 Current State **2**
1.1 Running Processes 2
1.2 Architectural Debt in the Supervisor 2
1.3 Collection Coverage Gap 2
1.4 Missing Per-Collection Configuration 2
1.5 Broken Annotation Pipeline 3
2 Collection Inventory **3**
2.1 Readiness Overview 3
2.2 Five Structural Families 5
2.2.1 Family 1 — Plain Scholarly 5
2.2.2 Family 2 — Dual-Tree Bilingual 5
2.2.3 Family 3 — Correspondence with Per-Letter XML 5
2.2.4 Family 4 — Correspondence with Batch-Prefixed Pages 5
2.2.5 Family 5 — Tablet and Transliteration 5
3 Target Architecture **6**
3.1 The Alias Collision Problem 6
3.2 Four-Strategy Edition Resolver 6
3.3 Architecture Diagram 7
3.4 Component Responsibilities 7
4 Configuration Layer **8**
4.1 Tier 1: editions.yaml Extension 8
4.2 Tier 2: collection.json Schema 8
4.2.1 collection.json Field Reference 8
4.3 Consolidating Scattered Config 9
5 Annotation Subsystem **10**
5.1 Design Principle 10
5.2 pdai_annotations.duckdb Schema 10
5.3 API Endpoints 12
5.4 Bridge Rule 12
6 Karman Absorption **12**
6.1 What Exists Today 12
6.2 Routes to Absorb 12
6.3 Support Modules to Move 13
6.4 UI Features viewer.html Lacks 14
6.5 OCR Monitor Generalisation 14
7 Hard Cutover **15**
7.1 Port Choice 15
7.2 Three Silently-Breaking Call Sites 15
7.3 File Edit Tiers 15
7.4 Implementation Sequence 17

1 Current State

1.1 Running Processes

Fourteen processes serve PDAI content permanently. Twelve of them are candidates for elimination. They are all children of `editions_supervisor.py` (pid 1596), which is itself managed by the `com.ai-platform.editions` launchd job.

Ten of the twelve are **byte-identical** `ai-viewer` instances: the same 6,038-line `pdai_server.py`, the same 70 HTTP routes, the same 10,060-line `viewer.html`. The only difference between them is the `--port` and `--registry` flags passed on launch:

- **5100** — enestroem (766 documents)
- **5103** — euler-opera (eulerOO + euler_sec, 100 documents)
- **5104** — kepler-gw (kepler-prim + kepler-sec, 44 documents)
- **5105** — copernicus (copernicus-prim + copernicus-sec, 64 documents)
- **5106** — waefler (waefler-prim + waefler-sec, 24 documents)
- **5107** — euler-oo-iv (eulerOO-IVA, 7 documents)
- **5108** — pirckheimer (pirckheimer + pirckheimer-sec, 2 documents)
- **5109** — kirch (kirch, 17 documents)
- **5112** — diaries (diaries, 10 documents)
- **5720** — sundials (sundial, 6 documents)

The two remaining retiring processes:

- **5111** — `ai-editions` (editions landing page, reads `editions.yaml`)
- **5725** — `ai-karman-manager` (separate FastAPI app, 17 routes, own static SPA)

Staying: **5113** (ai-query), **5890** (Glider semantic search).

1.2 Architectural Debt in the Supervisor

The supervisor reads a **hardcoded `EDITIONS` list** at lines 32–46 of `platform/scripts/editions_supervisor.py`. It does not read `config/editions.yaml`, which is consumed only by `ai-editions` on port 5111. Adding a new collection requires code changes, not config changes.

Each of the ten ai-viewer children loads its own Python interpreter and holds its own copy of every DuckDB connection, collection index, and instruction template in memory. Combined RSS for the ten ai-viewer processes is approximately 620 MB for serving content that differs only in which registry path they point to.

1.3 Collection Coverage Gap

There are 40 real collections on `/Volumes/PDAI-1/pdai-collections/` but only 14 edition registries. **24 collections have no viewer at all.** Of those 24, six are fully `search_ready` with a complete NLP pipeline (stanza, lemma index, ChromaDB, gazetteer):

- `causality`, `classical-philology`, `grasshoff`, `greek-astronomy`, `natural-philosophy`, `neugebauer`

These collections are indexed and queryable via ai-query, but there is no way for a user to browse their documents or view individual pages.

1.4 Missing Per-Collection Configuration

Zero `collection.json` files exist anywhere on PDAI-1. The server's `GET /api/collection/{collection}/meta` endpoint returns hardcoded defaults for only two collections (`enestroem` and `eulerOO`). Every other collection gets a generic empty response. This means:

- No collection-specific CSS theme colour (`--collection-primary` stays at the default Oxford blue)
- No display name, subtitle, or citation template per collection
- No declared page naming convention, structural family, or language defaults
- No panel allowlist (every panel shown regardless of whether the collection supports it)

1.5 Broken Annotation Pipeline

`viewer.html` line 7826 fires `POST /api/annotations` on its own port whenever the embedded `ai-query-chat` widget emits an `annotate` event. There is no such route in `pdai_server.py`. The failure is silently swallowed by `.catch(function(){})`. Annotations from scholarly reading sessions are lost at the source.

The ai-query server at port 5113 does implement `POST /api/annotations` (stored in `assessment.duckdb`), but the viewer never reaches it. A proxy route is the minimal fix; a proper document-scoped annotation store is the architectural fix described in Section 5.

---

2 Collection Inventory

2.1 Readiness Overview

Of 40 real collections, 17 are `search_ready`, one is `partial` (waefler-prim, chroma build incomplete), and 20 are `not_ready` due to missing NLP artifacts.

Table 1: Collection readiness and key artifacts

Collection	Docs	Ready	Stanza	Lemma	Chroma	Letters
enestroem	766	yes	yes	yes	yes	—
karman-nachlass	1 066	yes	yes	yes	yes	15 976
kepler-prim	26	yes	yes	yes	yes	—
kepler-sec	18	yes	yes	yes	yes	—
copernicus-prim	10	yes	yes	yes	yes	—
copernicus-sec	54	yes	yes	yes	yes	—
eulerOO	84	yes	yes	yes	yes	—
pirckheimer	1	yes	yes	yes	yes	354
pirckheimer-sec	1	yes	yes	yes	yes	—
waefler-prim	9	partial	yes	yes	—	—
waefler-sec	15	yes	yes	yes	yes	—
kirch	17	yes	yes	yes	yes	—
grasshoff	82	yes	yes	yes	yes	—
greek-astronomy	29	yes	yes	yes	yes	—
causality	12	yes	yes	yes	yes	—
classical-philology	1	yes	yes	yes	yes	—
natural-philosophy	1	yes	yes	yes	yes	—
neugebauer	6	yes	yes	yes	yes	—
sundial	6	no	—	—	—	—
diaries	11	—	—	—	—	—
euler_sec	16	no	—	—	—	—
assyrischeSammlung	3 710	no	—	—	—	—
ancient-library	1	no	—	—	—	—
ancient-metrology	1	no	—	—	—	—
ancient-philosophy-sec	4	no	—	—	—	—
ancient-waterclocks	1	no	—	—	—	—
architektur-antik	4	no	—	—	—	—
babylonian-astronomy	8	no	—	—	—	—
bernoulli-gw	1	no	—	—	—	—
early-modern-history-sec	6	no	—	—	—	—
economy	9	no	—	—	—	—
hama	5	no	—	—	—	—
history-of-biology-sec	3	no	—	—	—	—
medieval-sec	3	no	—	—	—	—
modern-physics	1	no	—	—	—	—
phototechnology	8	no	—	—	—	—
wissenschaftstheorie	12	no	—	—	—	—

Six collections have both a `<name>.duckdb` with an extended schema and a `concepts` DuckDB: `karman-nachlass`, `pirckheimer`, `euler00`, `kepler-prim`, `waefler-prim`, and `diaries` (with its own `diaries_lexicon.duckdb`). The `diaries` collection is structurally unique: it has no stanza corpus and no NLP pipeline, but carries a rich domain-specific schema covering calendar entries, tablets, transliterations, and plates.

2.2 Five Structural Families

Every collection belongs to one of five structural families that determine how `viewer.html` renders its pages. The unified server must carry family membership per collection, because rendering logic that works for plain scholarly text breaks for letter collections and is meaningless for tablets.

2.2.1 Family 1 — Plain Scholarly

Page XML follows `<page><main><p>`. No per-letter index, no bilingual mirror tree.

Collections: enestroem, waefler-prim, waefler-sec, sundial, grasshoff, greek-astronomy, causality, classical-philology, natural-philosophy, neugebauer, copernicus-prim, copernicus-sec, kirch, euler-sec, and all stub/not-ready collections.

Page naming: `pages/page_NNN/page_NNN.xml` (zero-padded, e.g. `page_001`).

2.2.2 Family 2 — Dual-Tree Bilingual

Kepler-prim carries a primary OCR tree at `xml/page_NNN.xml` alongside a translated tree at `xml_en/page_NNN.xml`, plus `pages/page_NNN/page_NNN.jpg` for facsimile. Each document has a `page.css` and `page.dtd` at the collection root. The viewer uses `PDAI_SCHEMA_RENDER = true` when these are present, activating the schema-aware XML-to-HTML renderer.

Collections: kepler-prim, and potentially kepler-sec (same page naming, no bilingual tree currently).

2.2.3 Family 3 — Correspondence with Per-Letter XML

Karman-nachlass generates a letter XML file for each extracted letter at `letters/letter-NNN.xml`. Letters are indexed in `karman-nachlass.duckdb` with 15,976 rows in `letters_resolved`, cross-referenced via `letter_persons`, `letter_places`, and `letter_institutions`. The viewer needs a correspondent index, A-Z letter strip, archive category filter, and per-page OCR quality verdict.

Collections: karman-nachlass.

2.2.4 Family 4 — Correspondence with Batch-Prefixed Pages

Pirckheimer uses a batch-prefixed page naming convention from its original OCR run: `pages/briefN__page_NNN/`. The collection-level `korrespondenz.xml` and `pirckheimer.duckdb` with 354 rows in `letters_resolved` provide the letter index. The viewer currently applies a fuzzy-lookup fix in `pdai_server.py` to handle the non-standard naming.

Collections: pirckheimer.

2.2.5 Family 5 — Tablet and Transliteration

The Astronomical Diaries collection has no stanza corpus. It uses `tablets.xml` at the document level and a domain-specific `diaries.duckdb` with tables `calendar`, `comments`, `lines`, `plates`, and `tablets`. Page XML follows the plain `<page><main><p>` structure, but document-level navigation must drive from the tablet index, not a manifest segment list.

Collections: diaries.

---

3 Target Architecture

3.1 The Alias Collision Problem

The current design assumes each edition runs on its own port, so alias resolution is trivial: the port determines the edition, and the alias is looked up in that edition's registry. This breaks under a single port.

The `euler-opera` and `euler-oo-iv` registries share two aliases: `978-3-319-33989-4` and `978-3-319-75942`. Both are present in `/Volumes/PDAI-1/editions/euler-opera/registry.json` and `/Volumes/PDAI-1/editions/euler-oo-iv/registry.json`. The proxy uses `strip_prefix: true`, so the unified server receives `/978-3-319-33989-4` with no edition context. Without edition resolution the server cannot know which registry to look in.

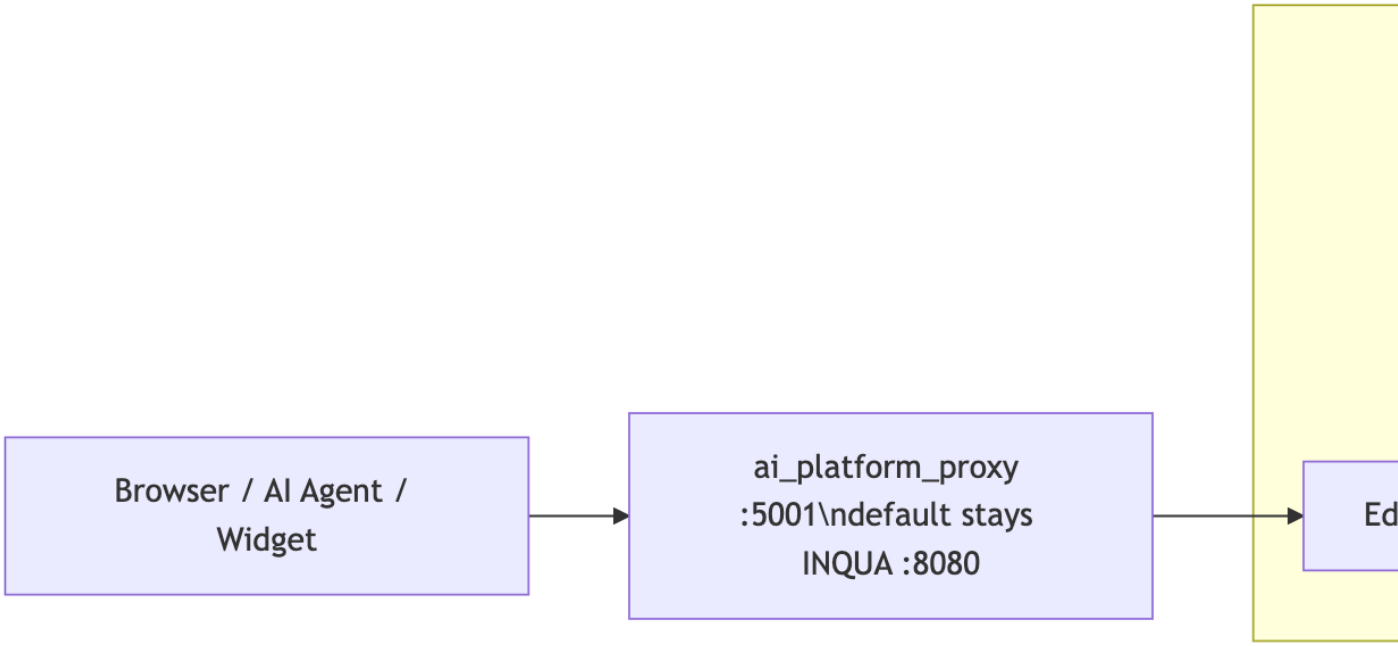
3.2 Four-Strategy Edition Resolver

The resolver applies four strategies in order, stopping on the first match:

1. **Host header.** The `Host` or `X-Forwarded-Host` header is matched against the `domain` field in `editions.yaml`. Requests from `kepler-gw.lettre.ai` resolve to the `kepler-gw` edition without any path inspection.
2. **Path prefix.** The request path is matched against `proxy_path` prefixes registered in `editions.yaml` (e.g. `/kepler-gw/`, `/karman/`). This covers requests arriving via the proxy's path-based routing.
3. **Collection segment.** Collection names are globally unique across all 40 collections. A request path of the form `/{collection}/{alias}` (e.g. `/kepler-prim/kgw-01`) uniquely identifies both the collection and the edition.
4. **Explicit override.** A `?edition=kepler-gw` query parameter forces the edition. This is the escape hatch for tools and tests that cannot set headers.

Every existing external URL continues to work: `routes.yaml` keeps all 12 path prefixes and all domain entries; only the `port:` value for each changes to the unified port. IONOS HTML frameset forwarding is unchanged.

3.3 Architecture Diagram



3.4 Component Responsibilities

Edition Registry Map. At startup the server loads all registries referenced by `editions.yaml` into memory. `PdaiPublicationServer` currently holds `self.documents` (one flat dict per process). The unified server replaces this with a dict keyed by edition id: `self.editions[edition_id].documents`. The `_primary_collection` per edition is derived from its registry `collections` list.

Config Layer. Reads `editions.yaml` for edition-level settings and `collection.json` from each collection root for per-collection settings. Falls back gracefully for collections without a `collection.json` (currently all of them). The existing `_COLLECTION_META_DEFAULTS` dict in `pdai_server.py:3418` is the transitional fallback until `collection.json` files are generated.

Capability Probes. Already implemented in ai-query (`has_letters_resolved()`, file-existence checks for `lemma_index.parquet`, `_lwgrammar.duckdb`, chroma directory). The unified server calls these at startup per collection and caches the results, exposing them via `GET /api/collection/{collection}/capabilities`. The `ai-query-chat` widget uses this to know which panels to show.

Annotation API. New endpoints `POST /api/annotations`, `GET /api/annotations`, `PATCH /api/annotations/{ann_id}`, `GET /api/snapshot/{id}`. Document-scoped annotations persist to `pdai_annotations.duckdb`; records carrying `phase` or `run_id` are bridged to ai-query's `assessment.duckdb` via proxy.

OCR Monitor. The existing Kepler-centric `/ocr-monitor/*` endpoints are generalised to be `ocr_catalog.yaml`-driven. The karman runner's `state.json` / `inflight.json` / `monitor.log` are wired in as a second catalog entry, fixing the current tmp-path mismatch (`karman-kimi-ocr` vs `karman-ocr`).

---

4 Configuration Layer

4.1 Tier 1: editions.yaml Extension

The current `editions.yaml` carries: `id`, `title`, `subtitle`, `port`, `proxy_path`, `domain`, `registry`, `collections`, optional `app`, `open_params`. All 11 edition entries need the `port` field removed (it is a server-side concern, not a config concern) and the following keys added:

```
# Example: kepler-gw edition - id : kepler-gw
title : Kepler Gesammelte Werke subtitle : Complete Works of Johannes Kepler
  proxy_path : /kepler-gw domain : kepler-gw.lettre.ai registry :
 /Volumes/PDAI-1/editions/kepler-gw/registry.json
  collections : - kepler-prim - kepler-sec setting : locked
: true initial_state : view : document # document | agents
  layout : both # facsimile | both | transcription lang : la
            # default language layer
  panel : none # none | query | dict | audit open_params : ""
  capabilities_required : - stanza - lemma
```

New keys:

- `setting.locked` — feeds `Setting(locked=True)` for the ai-query-chat widget. Always `true` for single-edition viewers.
- `initial_state` — the viewer URL state applied when opening the edition landing with no explicit params. Maps to `?view=`, `?lang=`, `?p=`, `?panel=`.
- `capabilities_required` — determines which panels are shown in the viewer. An edition without `stanza` hides the Query panel.

4.2 Tier 2: collection.json Schema

Each collection root receives a `collection.json` file. Currently none exist. The unified server reads this at startup and falls back to empty defaults. A generation script (`pdai-complete --generate-collection-json`) will create sensible defaults from the collection's existing status.json, stanza, and DuckDB state.

```
{ "display_name" :
 "Kepler Gesammelte Werke — Primary Sources" , "parent_name" : "kepler-gw" ,
  "structural_family" : "dual-tree-bilingual" , "page_naming" : "page_NNN"
, "langs" : [ "la" , "de" ] , "evidence_role" : "grounding" , "theme"
: { "primary_color" : "#002147" }, "alias_pattern" : "^kgw-
\\ d{2}$" , "citation_template" : "KGW {vol_nr}, p. {page_printed}" ,
"query" : { "language_default" : "la" , "aliases" : [ "kepler"
] , "doc_patterns" : [] , "lwgrammar_domains" : [ "astronomy" ,
 "natural_science" , "geometry" ] , "specialist" : "auto" },
"panels" : { "query" : true , "dict" : true , "audit" : true
, "agents" : false , "ocr_monitor" : true } }
```

4.2.1 collection.json Field Reference

Identity

- `display_name` — human-readable name shown in the browser tab, picker, and landing card
- `parent_name` — the edition id this collection belongs to (allows a collection to appear correctly even when loaded standalone)

- `structural_family` — one of `plain-scholarly`, `dual-tree-bilingual`, `correspondence-letter-xml`, `correspondence-batch-pages`, `tablet-transliteration`

Rendering

- `page_naming` — page directory pattern: `page_NNN` or `briefN__page_NNN` (batch-prefixed)
- `langs` — ordered list of language layers the viewer should offer (determines the `?lang=` dropdown)
- `theme.primary_color` — hex colour driving `--collection-primary` in `viewer.html`
- `alias_pattern` — regex that document aliases must match; shown as a hint in the picker

Query Settings

- `query.language_default` — the initial language passed to ai-query when no user preference is known; `la` for Latin corpora, `de` for German, `en` for English secondary literature
- `query.aliases` — list of natural-language names that refer to this collection (currently in `entity_link.yaml`; migrated here)
- `query.doc_patterns` — regex patterns for document alias recognition within queries (e.g. `^e\d{1,3}[a-z]?$` for enestroem)
- `query.lwgrammar_domains` — the `gaz_domain` values present in this collection's `_lwgrammar.duckdb`; used to populate the vocabulary scope selector
- `query.specialist` — `auto` (runtime probe), `correspondence-roster`, or `entity-roster`

Panels

The `panels` object is a per-collection allowlist. A panel set to `false` is hidden regardless of the edition's `capabilities_required`. This allows karman-nachlass to hide the Dictionary panel (no lwgrammar built) while keeping the Query panel, and allows the diaries to hide the Query panel entirely (no stanza corpus).

4.3 Consolidating Scattered Config

The following config currently lives outside any per-collection file and must be migrated into `collection.json` or kept in `editions.yaml`:

Currently in	Key	Migrate to
`pdai_server.py:3418` hard-coded defaults	enestroem, eulerOO meta	`collection.json`
`entity_link.yaml`	collection aliases	`collection.json query.aliases`
`entity_link.yaml`	doc patterns	`collection.json query.doc_patterns`
`evidence_roles.yaml`	grounding vs secondary	`collection.json evidence_role`
`context.py:268`	global `la` default	`collection.json query.language_default`
`pdai_server.py:598`	Kepler volume titles	`collection.json` per kgw-display_name NN
`pdai_server.py:631`	briefe aliases	`collection.json` for kirch/ query.aliases briefe

The `entity_link.yaml` and `evidence_roles.yaml` files remain authoritative during the transition: when `collection.json` does not exist (which is currently always), the server falls back to reading those YAML files exactly as today.

---

5 Annotation Subsystem

5.1 Design Principle

The viewer annotations are **document-scoped**: they anchor to a specific edition, collection, document alias, page index, and optionally a text span or sentence. This is a different concern from the pipeline annotations in `assessment.duckdb`, which anchor to a query run and a subagent phase for the purpose of improving instruction versions.

Both concerns are served. Document-scoped annotations persist to a new `pdai_annotations.duckdb`. Records that also carry a `run_id` or `phase` field (i.e. annotations made through the audit panel during a golden run) are additionally bridged to ai-query's `POST /api/annotations`, so they feed the existing learning loop.

All DuckDB connections are short-lived (open → operation → close), per the standing rule.

5.2 pdai_annotations.duckdb Schema

`annotation`

Column	Type	Notes
`ann_id`	`VARCHAR PK`	UUID
`ts`	`TIMESTAMP`	creation time
`person_id`	`VARCHAR`	nullable; links to ai-query's `persons` table
`edition`	`VARCHAR`	edition id from `editions.yaml`
`collection`	`VARCHAR`	collection name
`alias`	`VARCHAR`	document alias
`pidx`	`INTEGER`	page index (0-based); null for document-level
`target_kind`	`VARCHAR`	`document`, `page`, `span`, `sentence`, `letter`, `figure`
`target_ref`	`JSON`	kind-specific reference: `{"sent_idx": 4}` or `{"char_start": 120, "char_end": 145}`
`body`	`VARCHAR`	annotation text; markdown allowed
`tags`	`VARCHAR[]`	free-form tag list
`status`	`VARCHAR`	`open`, `resolved`, `archived`; default `open`
`parent_ann_id`	`VARCHAR`	nullable; for threaded replies

`annotation_revision`

Full edit history. Every PATCH to `annotation.body`, `tags`, or `status` appends a row here.

Column	Type	Notes
`rev_id`	`VARCHAR PK`	UUID
`ann_id`	`VARCHAR`	FK to annotation
`ts`	`TIMESTAMP`	revision time
`person_id`	`VARCHAR`	who made the change
`field`	`VARCHAR`	which field changed
`old_value`	`JSON`	previous value
`new_value`	`JSON`	new value

`annotation_snapshot`

A named, resolvable state that binds a document's viewer state to a set of annotations. Created by `POST /api/snapshot`. Retrieved by `GET /api/snapshot/{id}`. Used by agentic tasks to resume a reading session with all annotations intact.

Column	Type	Notes
`snapshot_id`	`VARCHAR PK`	short slug (e.g. `kepler-kgw01-p14-analysis`)
`ts`	`TIMESTAMP`	creation time
`person_id`	`VARCHAR`	creator
`edition`	`VARCHAR`	
`collection`	`VARCHAR`	
`alias`	`VARCHAR`	
`pidx`	`INTEGER`	page the viewer was on
`viewer_state`	`JSON`	full URL state: `{view, layout, lang, seg, panel}`
`annotation_ids`	`VARCHAR[]`	the set of annotation UUIDs in scope
`label`	`VARCHAR`	human-readable name for the snapshot
`notes`	`VARCHAR`	optional free-form description

`annotation_link`

Cross-references between document annotations and ai-query artefacts.

Column	Type	Notes
`ann_id`	`VARCHAR`	FK to annotation
`run_id`	`VARCHAR`	nullable; ai-query run UUID
`golden_id`	`VARCHAR`	nullable; golden benchmark UUID

Column	Type	Notes
`ts`	`TIMESTAMP`	time of link creation

5.3 API Endpoints

`POST /api/annotations` — create an annotation. Body: `{edition, collection, alias, pidx, target_kind, target_ref, body, tags, person_id}`. Returns the created `ann_id`. If the body also contains `phase` or `run_id`, the record is additionally proxied to `http://127.0.0.1:5113/api/annotations` (the existing ai-query learning-loop endpoint).

`GET /api/annotations?collection=&alias=&pidx=&status=` — list annotations. All parameters optional; without them returns the edition's entire annotation set.

`PATCH /api/annotations/{ann_id}` — update body, tags, or status. Creates a row in `annotation_revision`.

`POST /api/snapshot` — create a named snapshot. Body: `{edition, collection, alias, pidx, viewer_state, annotation_ids, label, notes, person_id}`. Returns `snapshot_id`.

`GET /api/snapshot/{snapshot_id}` — retrieve a snapshot with all its annotations expanded. Response includes `viewer_state` (ready to be injected as URL params) and `annotations[]` (full annotation objects). This is the agentic-task entry point: a task can call this endpoint with a known snapshot id and reconstruct the full reading context.

5.4 Bridge Rule

A record arriving at `POST /api/annotations` is stored locally in `pdai_annotations.duckdb` unconditionally. It is **additionally** forwarded to `http://127.0.0.1:5113/api/annotations` if and only if the request body contains a non-null `run_id` or `phase` field. This bridges the viewer's reading-session feedback into the query learning loop without coupling the two stores. The forward is fire-and-forget (no retry); failure of the bridge does not fail the viewer-side POST.

---

6 Karman Absorption

6.1 What Exists Today

`ai-karman-manager` is a separate Python package at `~/ai-mac-studio/scholarly/projects/ai-karman-manager/`, running under `uvicorn karman_manager.viewer.server:app` on port 5725. It has 17 live HTTP routes and a separate vanilla-JS SPA (`static/index.html`, 100 lines). It is started by `editions_supervisor.py` alongside the ten ai-viewer instances.

Its FastAPI app (`src/karman_manager/viewer/server.py`) makes an editable-install dependency on `ai-query` via `pyproject.toml`. This dependency becomes redundant when the unified server already depends on ai-query.

6.2 Routes to Absorb

Document and reading surface

Route	Purpose
`GET /api/summary`	Collection totals: docs, pages, correspondents, flagged count
`GET /api/categories`	Archive category list with document counts
`GET /api/correspondents`	Correspondents list filterable by letter and category
`GET /api/documents`	Document list with search, category, quality, letter, correspondent, and sort filters
`GET /api/documents/{doc_id}`	Single document with per-page verdict list
`GET /api/page/{doc_id}/{page_ref}`	Page metadata, raw XML, and parsed `<meta>`
`GET /api/page-html/{doc_id}/{page_ref}`	Rendered HTML transcript body
`GET /api/image/{doc_id}/{page_ref}`	Source JPEG

Correspondence-specific

Route	Purpose
`GET /api/content-status`	Whether `karman-nachlass.duckdb` exists, letter and entity counts
`GET /api/letters/{doc_id}`	Extracted letters: regest, sender, dates
`GET /api/entity-roster`	Persons, places, institutions ranked by letter count
`GET /api/letters-by-person/{ent_id}`	Letters mentioning a specific person entity

Query orchestration

Route	Purpose
`POST /api/resolve`	ai-query resolution via Conductor (default) or in-process fallback

All routes will be prefixed with the karman edition scope under the unified server. The `CollectionIndex` in-memory scan (60 s background rescan cycle via `index_builder.py`) is preserved.

6.3 Support Modules to Move

Module	Role	Generalizable?
`index_builder.py`	In-memory collection index: doc list, page verdicts, correspondent index	Yes — pattern applies to any correspondence collection
`doc_naming.py`	Karman archive category taxonomy, display-name humaniser	Karman-specific logic, but the pattern is reusable

Module	Role	Generalizable?
`ocr_quality.py`	Per-page XML quality classification (OK, LOW-CONF, EMPTY, BROKEN, SUSPECT)	Yes — generalises to any collection with OCR XML
`xml_render.py`	Page XML to HTML body renderer	Yes — same concern as ai-viewer's existing page rendering

These modules move into `ai-viewer/src/ai_viewer/collections/karman/`, keeping them co-located with the unified server and imported as needed.

6.4 UI Features viewer.html Lacks

The karman SPA provides six features that `viewer.html` does not implement. These must be ported before the karman UI is retired:

- **A-Z correspondent strip** with letter-index navigation, driven by `GET /api/correspondents`
- **Archive category filter** using karman's category taxonomy from `doc_naming.py`
- **Per-page OCR verdict chips** (OK / LOW-CONF / EMPTY / BROKEN / SUSPECT / UNPROCESSED) shown in the filmstrip and page header
- **Document quality badge** aggregating verdict counts per folder
- **"Needs review" filter** surfacing documents with flagged OCR pages
- **Archive box-order sort** using `DocMeta.sort_key` rather than registry order

These features are correspondence-family and OCR-pipeline concerns. They belong behind a feature flag in `viewer.html`: active only when `collection.json` sets `structural_family: correspondence-letter-xml` and `panels.ocr_monitor: true`.

6.5 OCR Monitor Generalisation

The karman OCR management surface must be **built**, not moved. The three HTTP endpoints that would serve live OCR status (`/api/status`, `/api/log/stream`, `/api/inflight`) were removed from `karman_manager/viewer/server.py` in an earlier development cycle. They survive only in stale documentation (`HOWTO.md §11`, `QUICKSTART.md`). The OCR pipeline is currently monitored by reading files from the filesystem CLI.

The existing Kepler-centric `/ocr-monitor/*` in `pdai_server.py` (lines 3941–4095) is the right foundation. It reads `ocr_catalog.yaml` to discover runners. There are two changes required:

1. **Catalog entry for karman.** Add a `karman-nachlass` entry to `ocr_catalog.yaml` pointing to `/Volumes/PDAI-1/tmp/karman-kimi-ocr/` (the actual path used by `async_ocr_pilot.py`). The current entry incorrectly points to `/Volumes/PDAI-1/tmp/karman-ocr/`.
2. **Generalise the state reader.** The current reader is shaped around Kepler's `kepler_ocr_run.py` output format. Replace the collection-specific path logic with a generic `state_dir` from the catalog entry, reading `state.json`, `inflight.json`, and the last N lines of `monitor.log` regardless of which collection's runner produced them.

The resulting `/ocr-monitor` dashboard becomes the single point of truth for active OCR runs across all collections.

---

7 Hard Cutover

7.1 Port Choice

The unified server runs on **port 5100**. This port is already canonical for ai-viewer in:

- `platform/config/routes.yaml` (primary `ai-viewer` route)
- `platform/registry/capabilities.json` (`required_ports`)
- `platform/manifest.json` (`ports: [5100]`)
- `agents/infrastructure/ai-cluster/src/cluster/config.py` (`"ai_viewer": 5100`)

All other per-edition ports (5103, 5104, 5105, 5106, 5107, 5108, 5109, 5111, 5112, 5720, 5725) are removed. The `routes.yaml` entries for those paths and domains remain; only their `port:` values change to 5100.

The **INQUA default** at port 8080 is unchanged, per the iron rule.

7.2 Three Silently-Breaking Call Sites

These break without any visible error and must be fixed before or simultaneously with the cutover:

1. `assemble.py` — `_build_collection_port_map()`

Currently discovers viewer ports by scanning `ps aux` for processes matching `ai-viewer --port N --registry`. Under a single server this yields one port (5100) for `enestroem` (the first registry registered), and empty results for everything else.

Fix: replace the `ps` scan with a static read of `editions.yaml`. For each edition, map its `collections[]` to port 5100 (the unified port). Alternatively, expose `GET /api/editions` on the unified server and have assemble.py call it at startup.

2. `editions_landing.py` — per-port TCP probes

The landing page sends TCP probes to each edition's individual port to determine whether it is alive, and renders status dots accordingly. Under a single server every probe except the 5100 one will fail (connection refused), showing all editions as down.

Fix: replace per-edition TCP probes with `GET http://127.0.0.1:5100/api/editions` and use the unified server's own health data, or probe `GET http://127.0.0.1:5100/api/edition/{edition_id}/health`.

3. `artifact.py` — hardcoded deep links

The `artifact.py` module hardcodes `http://localhost:5100/{collection}/...` as the pattern for constructing deep links to viewer pages. Since 5100 is the unified port, this partially works — but the URL structure may not align with the unified server's edition-prefixed routing. The links should be built from `editions.yaml` using the edition's `proxy_path`, not from a hardcoded port.

7.3 File Edit Tiers

Tier A — Runtime, must edit before cutover

File	Change
`platform/config/routes.yaml`	Change `port:` on all 12 retiring route entries to 5100
`platform/scripts/editions_supervisor.py`	Replace 12 spawns (10 ai-viewer + ai-editions + karman) with 1 unified server spawn

File	Change
`pdai/viewer/ai-viewer/config/editions.yaml`	Remove `port:` from all edition entries; add `initial_state`, `setting`, `capabilities_required`
`pdai/viewer/ai-viewer/src/ai_viewer/pdai_server.py`	Refactor for multi-edition registry map; add edition resolver
`pdai/viewer/ai-viewer/src/ai_viewer/editions_landing.py`	Replace per-port TCP probes with unified-server health API
`pdai/agents/ai-query/src/ai_query/pipeline/assemble.py`	Replace `ps` scan in `_build_collection_port_map()`
`pdai/agents/ai-query/src/ai_query/pipeline/artifact.py`	Replace hardcoded `localhost:5100/{collection}` links
`agents/infrastructure/ai-cluster/src/cluster/config.py`	`"ai_viewer": 5100` — already correct, confirm
`platform/config/repositories.yaml`	Update `port:` fields for each repository entry
`platform/registry/capabilities.json`	Update `required_ports` arrays; remove 5103–5112, 5720, 5725
`platform/manifest.json`	Update `ports` for ai-viewer
`platform/projects/registry.json`	Update per-project `ports.web`
`platform/config/service-commands.yaml`	Remove or update stale edition service entries
`development/ai-agent-orchestrator/scripts/kirch_golden_wait_and_verify.py`	Replace hardcoded `127.0.0.1:5109`
`pdai/viewer/ai-viewer/src/ai_viewer/cli.py`	Example ports in docstrings
`pdai/viewer/ai-viewer/src/ai_viewer/cli_stable.py`	Default port 5100 (already correct)
`pdai/collections/kepler-prim/scripts/ocr_monitor.py`	Comment reference to `:5104`
`scholarly/projects/ai-karman-manager/scripts/start_karman_viewer.sh`	`PORT=5725`

After route changes: run `ae-platform index` to rebuild `platform/catalog.duckdb` ports table.

Tier B — Skills and documentation, update for accuracy

File
`~/.cursor/skills/ai-platform-registry/SKILL.md`
`~/.cursor/skills/server-port-registry/SKILL.md`
`~/.cursor/skills/tailscale-network/SKILL.md`
`~/.cursor/skills/karman-nachlass-ocr/SKILL.md`
`~/.cursor/skills/visual-layout-audit/SKILL.md`
`platform/registry/skills/ai-platform-registry/SKILL.md`

File
`pdai/viewer/ai-viewer/README.md`
`pdai/viewer/ai-viewer/docs/VERSION_FEATURES.md`
`scholarly/projects/ai-karman-manager/HOWTO.md`
`scholarly/projects/ai-karman-manager/QUICKSTART.md`
`pdai/AGENTS.md`
`platform/AGENTS.md`

Tier C — Generated artefacts, re-render or leave

Quality reports and golden-run reports with baked-in `localhost:5104`, `localhost:5100` URLs. Not runtime-critical; re-render from live data after cutover.

7.4 Implementation Sequence

The following order minimises disruption. Each phase ends with live end-to-end probes before proceeding:

1. **Config layer.** Generate `collection.json` for all 17 search-ready collections. Extend `editions.yaml` with `initial_state`, `setting`, `capabilities_required`. No server changes yet.
2. **Edition-keyed server core.** Refactor `PdaiPublicationServer` to load all edition registries at startup into `self.editions`. Add the four-strategy edition resolver as a middleware. Run alongside existing processes on port 5099 for testing.
3. **Annotations.** Create `pdai_annotations.duckdb`, implement the five annotation endpoints and the bridge to ai-query, and add the proxy route for the widget's existing `POST /api/annotations`.
4. **Landing page integration.** Merge `editions_landing.py` into the unified server as a `GET /` route group. Retire port 5111.
5. **Karman absorption.** Move the 17 karman routes into the unified server under the karman edition scope. Port the karman-specific UI features into `viewer.html` behind the correspondence-family feature flag. Retire port 5725.
6. **OCR monitor generalisation.** Add the karman entry to `ocr_catalog.yaml`, generalise the state reader, verify against the live karman OCR runner.
7. **Hard cutover.** Stop all 12 retiring processes. Switch `editions_supervisor.py` to start one unified server on port 5100. Update `routes.yaml`. Run `ae-platform index`. Probe every edition end-to-end: host-header routing, deep-link resolution, query panel, audit panel, annotation POST.
8. **Cleanup.** Update all Tier B skills and documentation. Archive the karman-manager project.